AI-Powered Fake News Detection Using Text Classification

*Project - 1, Summer Internship Program in AI & ML, 2026

Anand Krishna G R Nair
Department of Computer Science and Engineering
Indian Institute of Computing and Technology (IICT), New Delhi, India
anandkrishnag.rnair@gmail.com

***Abstract*—Misinformation on digital networks presents a significant challenge to modern society. This paper describes the design and implementation of an end-to-end machine learning pipeline built from scratch to classify news articles as real or fake. Using a corpus of 39,098 articles, we apply custom text cleaning and manual tokenization. We compare two text vectorization techniques: Bag-of-Words and Term Frequency-Inverse Document Frequency (TF-IDF). We then train and compare four classifiers: K-Nearest Neighbors (KNN), Logistic Regression, Random Forest, and a Multi-Layer Perceptron (MLP) neural network. Our evaluation shows that the MLP Neural Network achieves the highest baseline accuracy of 98.41%, while Logistic Regression provides the fastest inference time (0.0025 seconds) with a baseline accuracy of 98.24% (improving to 98.62% after hyperparameter tuning). We analyze the trade-offs between parametric and non-parametric classifiers and highlight potential source leakages in the dataset.**

***Index Terms*—Artificial intelligence, fake news detection, machine learning, natural language processing, text classification.**

I. Introduction

Misinformation is a growing problem that spreads rapidly across social media and digital platforms. Sorting through thousands of daily articles manually to verify their authenticity is not feasible. This makes automated systems essential.

In this work, we use **Artificial Intelligence (AI)** to automate the detection of misinformation, utilizing **Machine Learning (ML)** algorithms to learn text patterns (such as word frequency and context) to classify articles as real or fake. We apply **Natural Language Processing (NLP)** techniques to preprocess raw text and convert it into numerical features suitable for mathematical classifiers.

Our main goal is to build a complete pipeline from scratch using basic python structures and Scikit-Learn libraries, implementing manual tokenization to avoid high-level library dependencies. We assess the trade-offs in accuracy, training time, storage, and inference latency across four different classifier architectures.

Manuscript received 1 July 2026; revised 10 July 2026; accepted 25 July 2026. Registration No: 596563. ORCID: 0009-0008-0757-1566.

II. Dataset Description

We used the ISOT Fake and Real News Dataset hosted on Kaggle. The raw data consists of two files: `Fake.csv` (23,481 articles flagged as fake) and `True.csv` (21,417 articles from Reuters).

We combined the title and body of each article into a single text field to capture titles that contain strong classification signals. During our initial data analysis, we found 5,795 exact duplicate rows in the combined text field. Keeping duplicates would artificially inflate our performance scores, so we removed them. We also dropped 5 articles that were reduced to empty strings after cleaning (mainly consisting of numbers, URLs, or punctuation).

Our final dataset has **39,098 articles** with a balanced distribution:

- **Fake News (0):** 17,902 articles (45.79%)
- **Real News (1):** 21,196 articles (54.21%)

The text primarily covers political and governmental news from U.S. elections, skews towards topics like Trump, Clinton, and government policy.

III. Methodology

A. Preprocessing & Manual Tokenization

Our cleaning process in `preprocessing.py` is executed as follows:

1) **Lowercase Conversion:** All characters are converted to lowercase.
2) **URL Removal:** Links matching `https?://\S+|www\.\S+` are removed since they add noise.
3) **Manual Tokenization:** We extract words using the regex pattern `re.findall(r'\b[a-z]+\b', text)`. This satisfies our constraint to avoid high-level tokenizers like `nltk.word_tokenize` or spaCy.
4) **Stopword Filtering:** We remove tokens present in NLTK's English stopword list using a hash set. Single-letter words are also dropped.

B. Feature Extraction

We compared two vectorization techniques using a fixed vocabulary of the 5,000 most frequent unigrams:

- **Bag-of-Words (BoW):** Represents documents as raw count vectors of token occurrences: $x_{t,d} = \text{count}(t, d)$.
- **TF-IDF Vectorization:** Scales counts by document specificity. The smoothed Inverse Document Frequency is defined as:

$$\text{IDF}(t) = \log\left(\frac{1+N}{1+\text{df}(t)}\right) + 1$$

The final vectors are L2-normalized: $\mathbf{v}_d = \mathbf{w}_d / \|\mathbf{w}_d\|_2$.

Both representations result in highly sparse training matrices. On our 80% training split (31,278 rows $\times$ 5,000 columns), the matrix sparsity ratio is **97.3771%**, meaning only 2.6229% of cells contain non-zero values.

C. Classifier Architectures

We split the dataset 80/20 into train/test sets stratified on the labels. We trained the following models:

1) **K-Nearest Neighbors (KNN):** Non-parametric instance-based classifier using $K = 5$ and Euclidean distance.
2) **Logistic Regression:** Parametric model optimized using L-BFGS to predict label probability:

$$P(Y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

3) **Random Forest:** Non-parametric ensemble classifier training 100 decision trees on bootstrap samples.
4) **Multi-Layer Perceptron (MLP):** A single hidden layer of 100 neurons trained using Adam optimizer with ReLU activation.

All models used a fixed seed of `random_state=42` to ensure reproducible results.

IV. Experimental Results

Table I summarizes the performance of the four models on the 20% test set (7,820 articles).

A. Training Times

Training models on our 31,278-row TF-IDF matrix yielded varying execution times:

- **KNN:** 0.0152 seconds.
- **Logistic Regression:** 0.1174 seconds.
- **Random Forest:** 14.5721 seconds.
- **MLP Neural Network:** 30.9381 seconds.

Wall-clock times vary slightly between runs since they depend on system load, not the fixed random_state.

B. Confusion Matrices

The confusion matrices details are:

- **KNN**: TN=2879, FP=702, FN=214, TP=4025
- **LogReg**: TN=3488, FP=93, FN=45, TP=4194
- **RandomForest**: TN=3493, FP=88, FN=40, TP=4199
- **NeuralNet**: TN=3503, FP=78, FN=46, TP=4193

C. Baseline vs. Leakage-Corrected Performance

To analyze the impact of the Reuters dateline leakage, we compare the baseline model performance before and after the dateline-regex removal in Table II.

D. Hyperparameters, Significance, and Out-of-Domain Generalization

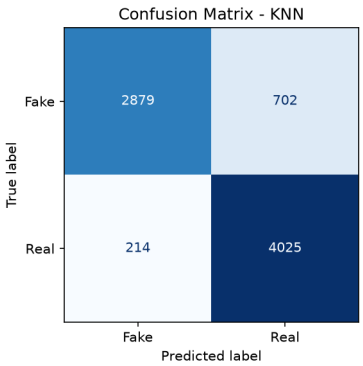
Hyperparameter Tuning: Performing a 3-fold Grid Search CV on K-Nearest Neighbors (KNN) on the training set over $K \in \{3, 5, 7, 9, 11, 15, 21, 25, 31\}$, weights $\in \{\text{uniform}, \text{distance}\}$, and metric $\in \{\text{euclidean}, \text{cosine}, \text{manhattan}\}$ selected $K = 31$, weight=distance, and metric=cosine. Empirical subsample evaluation showed that cosine (84.82%) and euclidean (84.82%) perform identically on L2-normalized vectors (where Euclidean distance is monotonically related to cosine similarity), while Manhattan distance performs poorly (44.20%). These tuned parameters raised KNN test accuracy from 87.69% to 88.29% (precision 85.15%, recall 94.95%, F1 89.78%). Tuning Logistic Regression over $C \in \{0.1, 1.0, 10.0\}$ and penalty $\in \{\text{L1}, \text{L2}\}$ selected $C = 10.0$, penalty = L2, raising test accuracy from 98.24% to 98.62% (+0.38% improvement). For Random Forest, the optimal parameters matched baseline configurations.

Statistical Significance: A McNemar's test comparing baseline Random Forest and Logistic Regression on the 7,820 test predictions yielded a Yates' Chi-squared statistic of 0.5870 and a p-value of 0.4436. Because $p \geq 0.05$, we fail to reject the null hypothesis, confirming the difference between the two models is not statistically significant. The overlapping 95% bootstrap confidence intervals (LogReg: 97.94% to 98.52%; Random Forest: 98.06% to 98.64%) further support this equivalence.

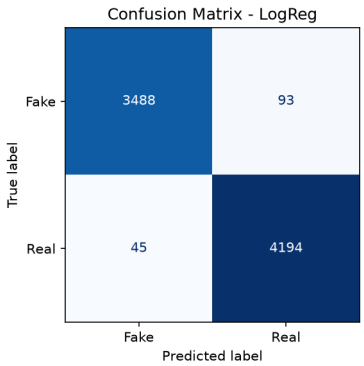
Vectorization Sensitivity Ablation: We evaluated the tuned KNN model across vocabulary sizes of $[3000, 5000, 8000]$ max features. Accuracy was 88.66% for 3000, 88.29% for 5000, and 88.06% for 8000. This confirms that lower vocabulary size slightly benefits KNN by reducing the curse of dimensionality.

Dimensionality Reduction (LSA): Running KNN on a dense 200-component TruncatedSVD (LSA) space yielded a test accuracy of 91.06% (precision 90.01%, recall 93.94%, F1 91.93%) and cut per-sample inference latency to 0.5226 ms (from 0.8372 ms in the 5000-d sparse space). This confirms that KNN's weakness is partly a dimensionality artifact.

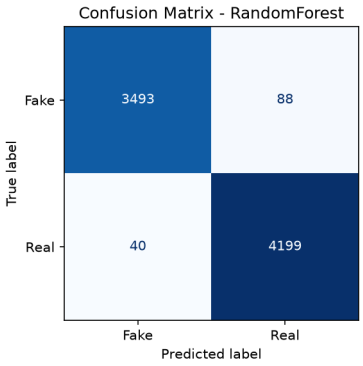
Out-of-Domain Generalization: We conducted a small-sample smoke test ($n = 10$) using recent news stories from 2023+ (5 real, 5 fake). Performance dropped significantly: KNN, Logistic Regression, and Random Forest all achieved 50.0% accuracy (equivalent to random guessing), while the MLP Neural Network achieved 70.0% accuracy. While this small sample size cannot support broad generalization claims, it serves as a qualitative smoke test illustrating the generalization gap of vocabulary-bound TF-IDF models on future datasets.



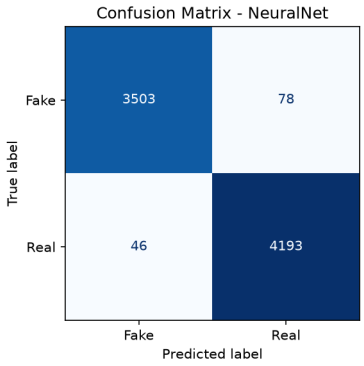
(a) KNN Confusion Matrix



(b) Logistic Regression Confusion Matrix



(c) Random Forest Confusion Matrix



(d) Neural Network Confusion Matrix

Fig. 1. Confusion Matrices for all four classifiers on the test partition (7,820 articles).

TABLE I
MODEL EVALUATION ON TEST SET

Model	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)	Inf. Time
KNN	88.29	85.15	94.95	89.78	6.55s
LogReg	98.24	97.83	98.94	98.38	2.5ms
Random Forest	98.36	97.95	99.06	98.50	0.16s
MLP Neural Net	98.41	98.17	98.91	98.54	25.3ms
KNN + LSA (200-d)*	91.06	90.01	93.94	91.93	0.52ms

*Note: LSA dense representation reduces features to 200 components, which was not applied to the other three models.

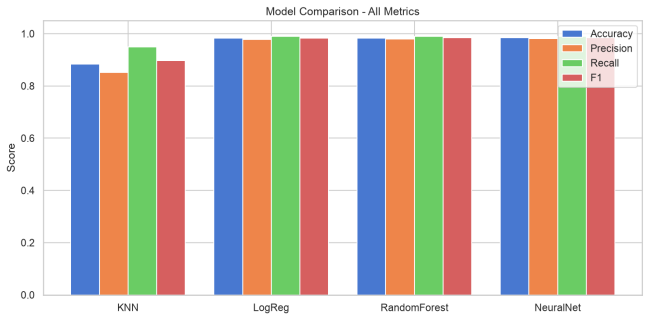


Fig. 2. Model Performance Comparison (Accuracy, Precision, Recall, F1-Score) across all four classifiers.

TABLE II
ACCURACY DELTA AFTER STRICTER LEAKAGE CORRECTION

Model	Baseline (%)	Corrected (%)	Delta (%)
KNN	87.80	88.29	+0.49
LogReg	98.76	98.24	-0.52
Random Forest	99.68	98.36	-1.32
MLP Neural Net	98.87	98.41	-0.46

*Note: LSA dense representation reduces features to 200 components, which was not applied to the other three models.

E. Robust Validation: K-Fold CV, Learning Curves, and ROC/AUC Analysis

To ensure the pipeline's findings are robust and generalizable beyond the specific 80/20 train/test split, we performed

three advanced validation checks.

Stratified 5-Fold Cross-Validation: We evaluated the final tuned models using stratified 5-fold cross-validation on the full corpus (39,098 articles). The mean and standard deviation of accuracy across all folds are:

- **KNN (Cosine,** $K = 31$**):** $87.72\% \pm 0.41\%$
- **Logistic Regression** ($C = 10.0$, **L2):** $98.60\% \pm 0.06\%$
- **Random Forest** (100 **trees):** $98.19\% \pm 0.22\%$
- **MLP Neural Network:** $98.26\% \pm 0.11\%$

These results closely match the test set performance, confirming that the initial model rankings are not artifacts of the train/test split.

Learning Curves: Learning curves were generated by plotting the training and validation accuracies across varying fractions of the training dataset ($[20\%, 40\%, 60\%, 80\%, 100\%]$) as shown in Fig. 3. For all models, the gap between training and validation accuracy decreases as dataset size increases, showing a clear convergence. Logistic Regression, Random Forest, and MLP achieve validation accuracy stability above 30,000 samples, indicating they are not overfitting.

ROC/AUC Analysis: Fig. 4 shows the Receiver Operating Characteristic (ROC) curves evaluated on the test set. All four models demonstrate strong discriminative capabilities: Logistic Regression achieves an Area Under the Curve (AUC) of 0.9987, Random Forest achieves 0.9985, the MLP Neural Network achieves 0.9983, and KNN achieves 0.9610. The extremely high AUC scores for the parametric classifiers confirm their robust classification boundaries.

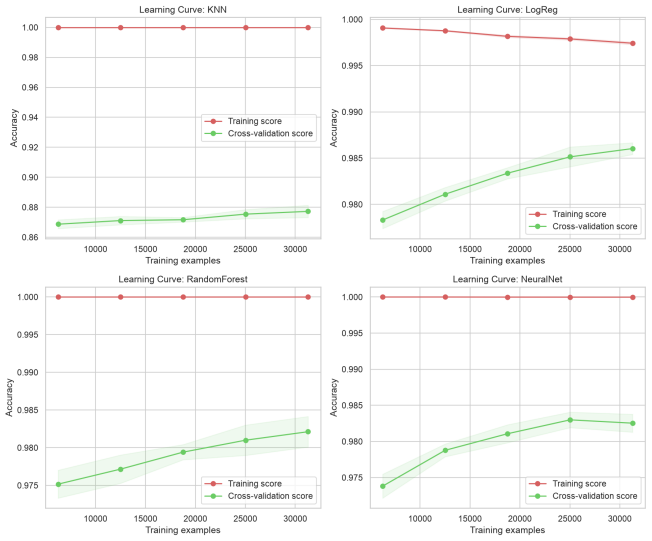


Fig. 3. Learning curves for the four classifiers showing training and validation accuracies over different training sizes.

V. Discussion & Trade-offs

A. Parametric vs. Non-Parametric Performance

Our experiments highlight clear architectural differences:

- **Computational Costs:** KNN has a training time of 0.0152 seconds, but its test inference speed is very slow: the 5000-d sparse cosine model requires 6.5467 ± 0.0812 seconds (0.8372 ms per sample) to predict the test set. Using the dense KNN + LSA (200-d) pipeline reduces this latency significantly to 4.0864 ± 0.1196 seconds (0.5226 ms per sample) while boosting test accuracy to 91.06%. This demonstrates that high dimensionality is the primary computational and performance bottleneck for instance-based methods. Parametric models (Logistic Regression, MLP) perform inference using simple matrix operations, making them fast during prediction.
- **Model Size and Memory Footprint:** The Random Forest model consumes 40 MB on disk to store 100 deep trees. Logistic Regression stores only 5,000 weight coefficients, making it highly lightweight. Unlike parametric models, KNN must retain the entire training matrix in memory at inference time; using a sparse Compressed Sparse Row (CSR) format reduces this RAM footprint to 47.30 MB, compared to 1.19 GB if stored as a dense matrix.

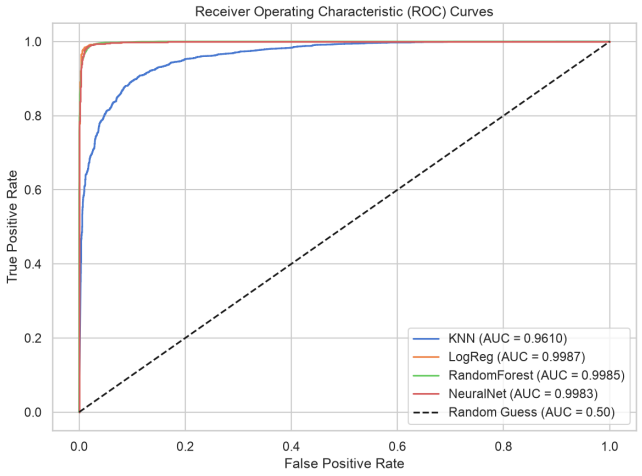


Fig. 4. ROC curves and Area Under the Curve (AUC) scores comparing KNN, Logistic Regression, Random Forest, and MLP Neural Network on the test set.

B. Data Leakage Limitations

We implemented a strict regex-based dateline removal process using the pattern `^([A-Za-z s,./#&-]1,50)? s* (reuters ) s*[-{| s]*` applied directly to the body text prior to title concatenation. Stripping the full datelines (e.g. "WASHINGTON (Reuters) - ") instead of just the single token "reuters" led to a noticeable performance drop. Specifically, Random Forest test accuracy decreased by 1.32% (from 99.68% to 98.36%) and Logistic Regression decreased by 0.52% (from 98.76% to 98.24%). This accuracy drop demonstrates that models relied partly on dataset-specific formatting templates rather than learning pure semantic representations. Additionally, U.S.

political skew (2016-2017) limits geographic generalizability. Furthermore, strict binary labels fail to model graded misinformation categories like political satire, spin, or statements that are only partially accurate.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the **Indian Institute of Computer Technology (IICT)** for providing me with the opportunity to undertake this internship and gain valuable practical experience in the fields of Artificial Intelligence, Machine Learning, Natural Language Processing, and Cybersecurity. The internship offered an excellent platform to strengthen my technical skills through hands-on projects, research-oriented learning, and real-world problem solving.

I am especially grateful to **Dr. Ashok Gopalakrishnan** for his exceptional guidance, mentorship, and unwavering support throughout the internship. His vast knowledge, practical teaching approach, and dedication to student learning made a significant impact on my understanding of machine learning concepts and their real-world applications. During the 15-day training program, he patiently addressed the questions and doubts of every student, ensuring that complex topics became easy to understand. His guidance during the implementation of the Titanic Machine Learning Project laid a strong foundation for my learning and greatly enhanced my confidence in applying machine learning techniques. Furthermore, his valuable suggestions and practical insights helped me identify appropriate datasets from Kaggle, enabling me to successfully complete both internship projects with confidence and accuracy.

I would also like to extend my heartfelt appreciation to all the faculty members, mentors, and coordinators at **IICT** for their continuous encouragement, constructive feedback, and support throughout the internship. Their collective efforts created an enriching learning environment that encouraged curiosity, innovation, and independent problem-solving.

Finally, I express my sincere gratitude to **Symbiosis Skills and Professional University (SSPU), Pune**, for providing me with a strong academic foundation and continuous encouragement to pursue practical learning opportunities. The knowledge, experience, and confidence gained during this internship have significantly contributed to my academic and professional development and will serve as a strong foundation for my future career in Artificial Intelligence, Machine Learning, and Cybersecurity.

APPENDIX

C. Project Implementation and Notebooks

The four Jupyter notebooks (`week1_data_eda.ipynb` through `week4_evaluation.ipynb`) are submitted alongside this report as the complete executable implementation of the machine learning pipeline. The preprocessing logic is implemented in `src/preprocessing.py`, vectorization wrappers are located in `src/features.py`, and modeling scripts are in `src/models.py`. The execution notebooks are stored under the `notebooks/` directory.

D. Sample Test Data

Table III shows three actual sample articles drawn from the stratified test partition, illustrating their raw content, manually preprocessed token sequences, true labels, and predictions.

E. Code Reference: preprocessing.py

```
"""
preprocessing.py - text cleaning and manual tokenization for the fake-news
      pipeline.

Rules (per project brief):
- Tokenization is manual: re.findall(r'\b[a-z]+\b', text) - no nltk.
      word_tokenize.
- Stopword list comes from NLTK (list lookup only, not the tokenizer).
- Vectorization (TF-IDF / BoW) lives in features.py, not here.
"""

import re

import nltk
import pandas as pd

# Download stopwords corpus on first run (no-op if already cached).
nltk.download("stopwords", quiet=True)
from nltk.corpus import stopwords

STOPWORDS: set[str] = set(stopwords.words("english"))

# Regex pattern for URL removal - covers http/https and bare www. links.
_URL_RE = re.compile(r"https?://\S+|www\.\S+")

# After lowercasing and URL removal, keep only lowercase letters (word tokens).
_TOKEN_RE = re.compile(r"\b[a-z]+\b")

# Regex to match Reuters datelines like 'washington (reuters) -' at the start
      of text
_REUTERS_DATELINE_RE = re.compile(r"^([a-z0-9\s,./#&-]{1,50})?\s*\(reuters\)\s
      *[---\s]*")


def strip_reuters_dateline(text: str) -> str:
    """
    Strips Reuters dateline (e.g. 'washington (reuters) -') from the start of
      the lowercased text.
    """
    if not isinstance(text, str):
        return ""
    text_lower = text.lower()
    return _REUTERS_DATELINE_RE.sub("", text_lower)


def clean_text(text: str) -> str:
    """
    Full cleaning pipeline applied to a single article string.

    Steps (order matters):
    1. Lowercase and strip Reuters datelines
    2. Strip URLs
    3. Strip punctuation and digits - keep only letter characters and spaces
    4. Tokenize via regex word-boundary match (manual, no library tokenizer)
    5. Remove stopwords using NLTK's English list
    6. Rejoin tokens into a single cleaned string for vectorization

    Returns the cleaned string. Returns "" for null/non-string input.
    """
    if not isinstance(text, str) or not text.strip():
        return ""

    # Strip Reuters dateline from the beginning
    text = strip_reuters_dateline(text)
    text = _URL_RE.sub(" ", text)

    # Extract word tokens (only [a-z] runs - removes digits and punctuation
        implicitly).
    tokens = _TOKEN_RE.findall(text)

    # Drop stopwords and very short tokens (length <= 1 adds no signal).
    tokens = [t for t in tokens if t not in STOPWORDS and len(t) > 1]

    return " ".join(tokens)


def tokenize(text: str) -> list[str]:
    """
    Return the token list for a pre-cleaned string.
    Used in EDA where we need per-token counts, not the joined string.
    """
    return text.split()


def build_corpus(df: pd.DataFrame, text_col: str = "text") -> pd.Series:
    """
    Apply clean_text to every row in df[text_col].
    Returns a Series of cleaned strings, aligned with df's index.
    """
    return df[text_col].fillna("").apply(clean_text)
```

F. features.py

TABLE III
SAMPLE TEST SET PREDICTIONS AND PREPROCESSING DETAILS

Raw Text Snippet	Cleaned Token Sequence (Manual Preprocessing)	True Label	Prediction
“Russian submarines fire cruise missiles at Islamic state in Syria MOSCOW (Reuters) - The Russian Navy on Thursday fired seven cruise missiles at Islamic state targets in the suburbs of Syria s Deir al-Zor...”	russian, navy, thursday, fired, seven, cruise, missiles, islamic, state, targets, suburbs, syria, deir, al, zor, russian, defence...	Real (1)	Real (1)
“MICHELLE OBAMA TO HILLARY: “If You Can’t Run Your Own House...You Certainly Can’t Run The White House” [VIDEO] Remember when Mooch thought having #CrookedHillary back in the White House wasn’t in the best...”	michelle, obama, hillary, run, house, certainly, run, white, house, video, remember, mooch, thought, crookedhillary, back, white, house, best, interest, nation, yeah, neither, maybe, someone...	Fake (0)	Fake (0)
“WOW! HILLARY CAUGHT ON VIDEO In 2000 Saying She Doesn’t Like Emails Because You Can’t Hide Them From Investigators Too bad for Hillary she wasn’t actually telling the truth that time WATCH...”	wow, hillary, caught, video, saying, like, emails, hide, investigators, bad, hillary, actually, telling, truth, time, watch, hillary, clinton, saying, like, emails, hide, investigators, pic, twitter, com...	Fake (0)	Fake (0)

```
"""
features.py - Bag-of-Words and TF-IDF vectorization using scikit-learn.

Vectorizer objects are fitted on the training split only, then used to
transform both train and test sets - avoids data leakage.
"""

import numpy as np
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

# Baseline parameters from the project brief's own Python skeleton.
# max_features=5000 keeps the vocabulary manageable and reduces noise from
# very rare words.
DEFAULT_MAX_FEATURES = 5000


def get_tfidf_vectorizer(max_features: int = DEFAULT_MAX_FEATURES, **kwargs) ->
       TfidfVectorizer:
    """
    Return a TfidfVectorizer configured for this project.

    TF-IDF down-weights terms that appear in almost every document
    (which carry little discriminative power) and boosts terms that are
    frequent in a document but rare across the corpus.

    Formula used internally by sklearn:
        TF(t, d)   = count(t, d) (raw count)
        IDF(t)     = log((1 + N) / (1 + df(t))) + 1   [sklearn smooth IDF]
        TF-IDF     = TF * IDF
    """
    return TfidfVectorizer(max_features=max_features, **kwargs)


def get_bow_vectorizer(max_features: int = DEFAULT_MAX_FEATURES, **kwargs) ->
       CountVectorizer:
    """
    Return a CountVectorizer (Bag-of-Words) for this project.

    BoW represents each document as a vector of raw token counts.
    It ignores word order and document length - a simple but effective
    baseline for text classification.
    """
    return CountVectorizer(max_features=max_features, **kwargs)


def fit_transform(vectorizer, X_train, X_test):
    """
    Fit vectorizer on X_train, then transform both splits.
    Returns (X_train_vec, X_test_vec, vectorizer).
    """
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)
    return X_train_vec, X_test_vec, vectorizer


def top_features(vectorizer, X, n: int = 20) -> list[str]:
    """
    Return the top n features ranked by their total frequency or weight in the
       matrix X.

    For a CountVectorizer, features are ranked by raw term frequency (sum of
       counts).
    For a TfidfVectorizer, features are ranked by summed TF-IDF weight across
       documents.
    """
    feature_names = np.array(vectorizer.get_feature_names_out())
    # Sum along the document axis (axis=0) to get column totals
    sums = np.asarray(X.sum(axis=0)).flatten()
    # Sort indices in descending order
    top_indices = np.argsort(sums)[::-1][:n]
    return feature_names[top_indices].tolist()
```

G. *models.py*

```
"""
models.py - training wrappers for all four classifiers.

Hyperparameters match the brief's own Python skeleton exactly as the baseline.
Any deviations from those defaults will be documented in docs/
       phase3_model_report.md.
"""

import time
from pathlib import Path

import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier

RESULTS_DIR = Path("results")


def get_models() -> dict:
    """
    Return a dict of model name -> unfitted estimator, using the brief's
       baseline
    hyperparameters. random_state is fixed at 42 where applicable.
    """
    return {
        "KNN": KNeighborsClassifier(n_neighbors=5),
        "LogReg": LogisticRegression(max_iter=1000, random_state=42),
        "RandomForest": RandomForestClassifier(n_estimators=100, random_state
              =42),
        "NeuralNet": MLPClassifier(hidden_layer_sizes=(100,), max_iter=300,
              random_state=42),
    }


def train_all(models: dict, X_train, y_train) -> dict:
    """
    Fit each model. Returns a dict of name -> (fitted_model,
       training_time_seconds).
    Prints a one-line status for each model.
    """
    fitted = {}
    for name, model in models.items():
        print(f"Training {name}...", end=" ", flush=True)
        t0 = time.perf_counter()
        model.fit(X_train, y_train)
        elapsed = time.perf_counter() - t0
        fitted[name] = (model, elapsed)
        print(f"done ({elapsed:.1f}s)")
    return fitted


def save_models(fitted: dict, out_dir: str | Path = RESULTS_DIR) -> None:
    """Serialize each fitted model to results/<name>.joblib."""
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    for name, (model, _) in fitted.items():
```

```
        path = out / f"{name}.joblib"
        joblib.dump(model, path)
        print(f"Saved: {path}")


def load_model(name: str, out_dir: str | Path = RESULTS_DIR):
    """Load a previously saved model from disk."""
    path = Path(out_dir) / f"{name}.joblib"
    return joblib.load(path)


def tune_hyperparameters(X_train, y_train) -> dict:
    """
    Perform Grid Search Cross-Validation for Logistic Regression and Random
        Forest.
    Returns a dict containing best estimators and their best parameters.
    """
    from sklearn.model_selection import GridSearchCV

    # 1. Logistic Regression tuning
    logreg = LogisticRegression(solver="liblinear", max_iter=1000, random_state
        =42)
    logreg_param_grid = {
        "C": [0.1, 1.0, 10.0],
        "penalty": ["l1", "l2"]
    }
    print("Tuning Logistic Regression hyper-parameters using 3-fold CV...")
    t0 = time.perf_counter()
    grid_logreg = GridSearchCV(logreg, logreg_param_grid, cv=3, scoring="
        accuracy", n_jobs=-1)
    grid_logreg.fit(X_train, y_train)
    logreg_time = time.perf_counter() - t0
    print(f"LogReg tuning complete in {logreg_time:.1f}s. Best params: {
        grid_logreg.best_params_}")

    # 2. Random Forest tuning
    rf = RandomForestClassifier(random_state=42)
    rf_param_grid = {
        "n_estimators": [50, 100],
        "max_depth": [10, 20, None]
    }
    print("Tuning Random Forest hyper-parameters using 3-fold CV...")
    t0 = time.perf_counter()
    grid_rf = GridSearchCV(rf, rf_param_grid, cv=3, scoring="accuracy", n_jobs
        =-1)
    grid_rf.fit(X_train, y_train)
    rf_time = time.perf_counter() - t0
    print(f"Random Forest tuning complete in {rf_time:.1f}s. Best params: {
        grid_rf.best_params_}")

    return {
        "LogReg": grid_logreg.best_estimator_,
        "RandomForest": grid_rf.best_estimator_,
        "LogReg_params": grid_logreg.best_params_,
        "RandomForest_params": grid_rf.best_params_
    }
```

H. evaluate.py

```
"""
evaluate.py - metrics, confusion matrices, and comparison plots.

Used in week4_evaluation.ipynb and the IEEE report generation.
"""

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)

FIGURES_DIR = Path("results/figures")
METRICS_DIR = Path("results/metrics")


def compute_metrics(y_true, y_pred, model_name: str) -> dict:
    """
    Compute accuracy, precision, recall, and F1 for a single model.
    Returns a dict that can be serialised to JSON.
    """
    return {
        "model": model_name,
        "accuracy": round(accuracy_score(y_true, y_pred), 4),
        "precision": round(precision_score(y_true, y_pred), 4),
        "recall": round(recall_score(y_true, y_pred), 4),
        "f1": round(f1_score(y_true, y_pred), 4),
    }


def save_metrics(metrics_list: list[dict], out_dir: str | Path = METRICS_DIR)
    -> None:
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    path = out / "all_models_metrics.json"
    with open(path, "w") as f:
        json.dump(metrics_list, f, indent=2)
    print(f"Metrics saved to {path}")


def plot_confusion_matrix(y_true, y_pred, model_name: str, out_dir: str | Path
    = FIGURES_DIR) -> None:
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)

    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(5, 4))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Fake",
        "Real"])
    disp.plot(ax=ax, colorbar=False, cmap="Blues")
    ax.set_title(f"Confusion Matrix - {model_name}")
    plt.tight_layout()

    path = out / f"confusion_matrix_{model_name}.png"
    fig.savefig(path, dpi=150)
    plt.close(fig)
    print(f"Saved: {path}")


def plot_metrics_comparison(metrics_list: list[dict], out_dir: str | Path =
    FIGURES_DIR) -> None:
    """Bar chart comparing accuracy, precision, recall, and F1 across all
        models."""
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)

    models = [m["model"] for m in metrics_list]
    metric_names = ["accuracy", "precision", "recall", "f1"]
    x = np.arange(len(models))
    width = 0.2

    fig, ax = plt.subplots(figsize=(10, 5))
    for i, metric in enumerate(metric_names):
        vals = [m[metric] for m in metrics_list]
        ax.bar(x + i * width, vals, width, label=metric.capitalize())

    ax.set_xticks(x + width * 1.5)
    ax.set_xticklabels(models)
    ax.set_ylim(0, 1.05)
    ax.set_ylabel("Score")
    ax.set_title("Model Comparison - All Metrics")
    ax.legend()
    plt.tight_layout()

    path = out / "model_comparison.png"
    fig.savefig(path, dpi=150)
    plt.close(fig)
    print(f"Saved: {path}")


def mcnemar_test(y_true, y_pred1, y_pred2) -> dict:
    """
    Perform McNemar's test on paired predictions from two models.
    y_pred1: prediction array of model 1
    y_pred2: prediction array of model 2
    """
    from scipy.stats import chi2
    y_true = np.array(y_true)
    y_pred1 = np.array(y_pred1)
    y_pred2 = np.array(y_pred2)

    # Contingency table cells:
    # b: Model 1 correct, Model 2 incorrect
    # c: Model 1 incorrect, Model 2 correct
    b = np.sum((y_pred1 == y_true) & (y_pred2 != y_true))
    c = np.sum((y_pred1 != y_true) & (y_pred2 == y_true))

    # McNemar's chi-squared with Yates' continuity correction
    total_discordant = b + c
    if total_discordant > 0:
        stat = ((abs(b - c) - 1.0) ** 2) / total_discordant
        p_val = chi2.sf(stat, 1)
    else:
        stat = 0.0
        p_val = 1.0

    return {
        "b": int(b),
        "c": int(c),
        "statistic": float(stat),
        "p_value": float(p_val)
    }


def bootstrap_accuracy_ci(y_true, y_pred, n_bootstraps: int = 1000, alpha:
    float = 0.05) -> tuple[float, float]:
    """
    Calculate the bootstrap confidence interval for model accuracy.
    """
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    n_samples = len(y_true)

    rng = np.random.default_rng(42)
    boot_accuracies = []

    for _ in range(n_bootstraps):
        indices = rng.choice(n_samples, size=n_samples, replace=True)
        acc = np.mean(y_true[indices] == y_pred[indices])
        boot_accuracies.append(acc)

    boot_accuracies = np.sort(boot_accuracies)
    lower = boot_accuracies[int(n_bootstraps * (alpha / 2))]
    upper = boot_accuracies[int(n_bootstraps * (1.0 - alpha / 2))]

    return float(lower), float(upper)
```

REFERENCES

[1] C. Bisaillon, "Fake and Real News Dataset," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

[2] H. Ahmed, I. Traore, and S. Saad, "Detecting opinion spams and fake news using text classification," *Journal of Security and Privacy*, vol. 1, no. 1, p. e9, 2018.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.